

2110531 - Data Science and Data Engineering Tools

Data Lakehouse Case Study: MetroRide

Asst. Prof. Natawut Nupairoj, Ph.D.

Department of Computer Engineering, Chulalongkorn University

natawut.n@chula.ac.th

CHULA ENGINEERING
Foundation toward Innovation

Outline

1

DuckLake concepts

- Catalog vs. data files
- Layers as schemas
- Snapshots and transactions
- Data inlining

No code

2

pyducklake API

- Catalog → Table → Scan
- Define, write, read
- Snapshots, time travel, changes
- SQL patterns we reuse

`step0_pyducklake_primer.ipynb`

3

Tutorial

- Create bronze and silver
- Ingest and transform April
- Append May without duplicates
- Time travel in silver

`tutorial.ipynb`

Fictional Case Study: The MetroRide



A fictional Bangkok ride-hailing service sends one nightly CSV export from its booking system.

“How many requests were completed, and how long did passengers wait to be picked up?”

Operations manager

- trips_2026-04.csv and trips_2026-05.csv
- Timestamps in Bangkok time (+07:00)
- Blank cell = value absent (e.g. fare of a cancelled trip)

2

monthly files

269

trips (132 + 137)

4

service zones

10

raw columns, all text

CHULA ENGINEERING
Foundation toward Innovation

Part 1

DuckLake concepts

Where data and metadata live, and how every change is recorded



A DuckLake table lives in two places

Catalog

`metadata.ducklake`

- Schemas, tables, columns
- The list of data files per table
- Snapshots and commit messages
- A SQL database: DuckDB, SQLite, PostgreSQL or MySQL

points to

Data files

`data/`

```
data/bronze/trips_raw/
  ducklake-01a1….parquet
data/silver/trips/
  ducklake-01a1….parquet
```

Immutable Parquet files: written once, never edited

Iceberg and Delta keep metadata as files. DuckLake keeps it in a database, so a commit is one SQL transaction and metadata can be queried with plain SQL.

Medallion layers are DuckLake schemas

bronze

`bronze.trips_raw`

- Values exactly as received
- Every source column is text
- + source_file, source_row_number, file_sha256, ingested_at

silver

`silver.trips`

- Typed: timestamptz, decimal, date
- Validated by phase-1 rules
- One row per trip_id + service_date, time_slot

gold

`gold.daily_service_performance`

- KPIs per date and time slot
- Built in a later exercise
- Not part of this tutorial

Rule of thumb: bronze never rejects anything; silver never accepts anything unchecked.

Every commit creates a snapshot



Snapshot ids only grow

Each commit (create, append, delete) adds one.

Snapshots are catalog-wide

One id sequence for all tables, so “latest” means latest in the catalog.

Old files are never changed

Any snapshot can be read again later: time travel.

CHULA ENGINEERING
Foundation toward Innovation

Transactions group writes; inlining hides small ones

Transactions

- Several writes become one atomic commit and one snapshot
- Error inside the block: everything is rolled back
- A commit can carry an author and a message

```
with catalog.begin_transaction() as txn:
    catalog.set_commit_message(
        "bronze: ingest April")
    txn.load_table(t).append(rows)
```

Data inlining

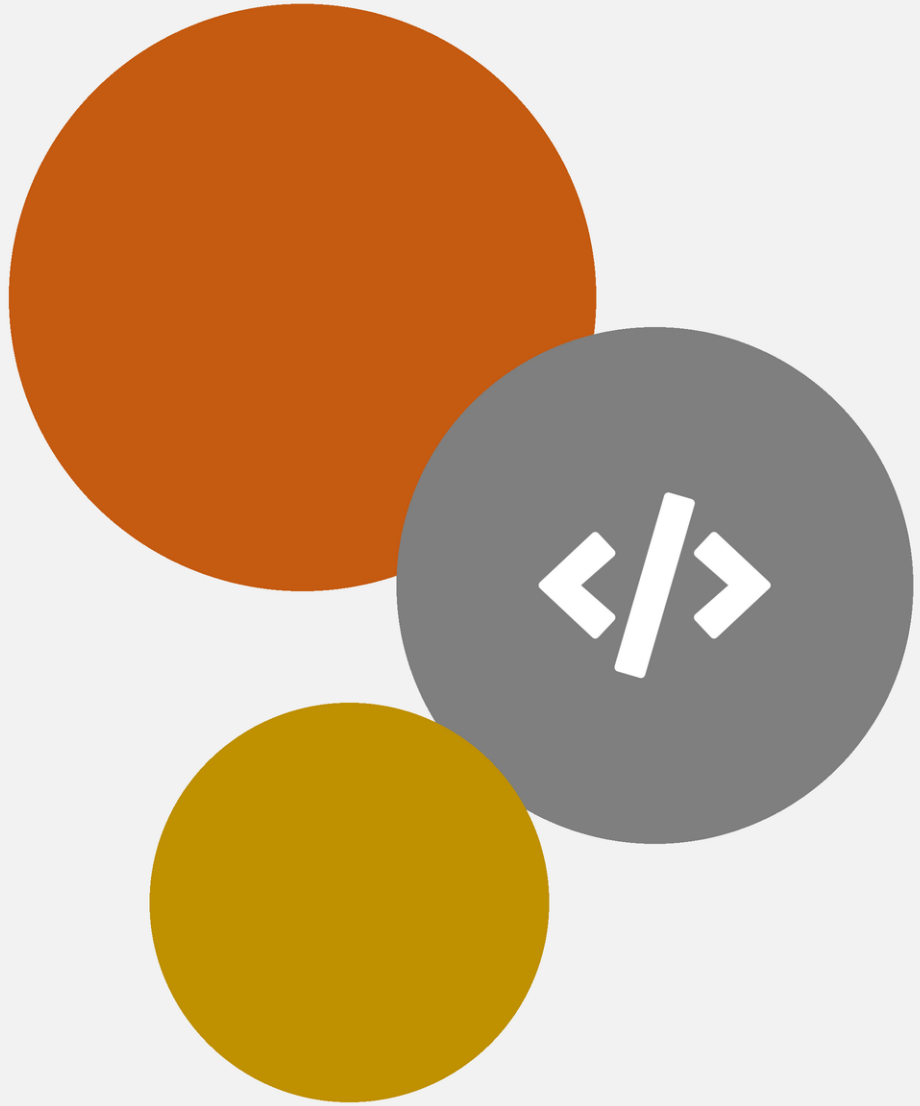
- Small inserts are stored inside the catalog database, not as Parquet
- Avoids thousands of tiny files when data arrives often
- We switch it off in class so each insert is a visible file

```
catalog.set_option(
    "data_inlining_row_limit", "0")
```

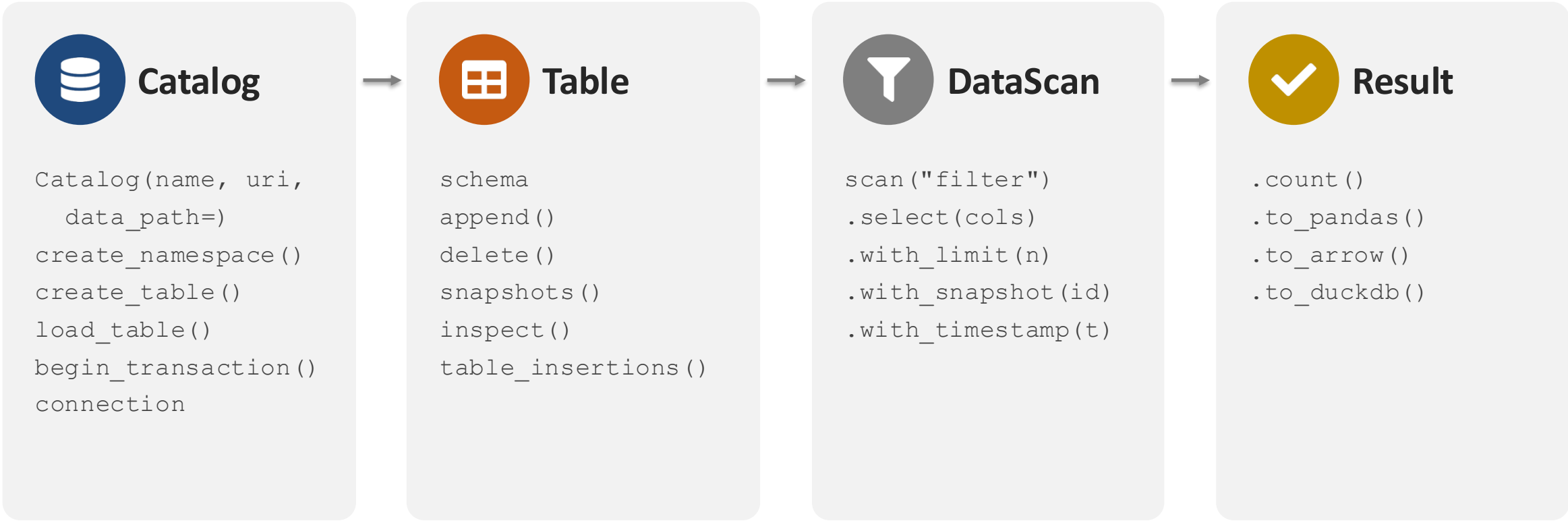
Part 2

The pyducklake API

A pyiceberg-style Python client for DuckLake, with DuckDB underneath



Four objects to remember



Nothing is read until a Result method runs. Each DataScan method returns a new scan, so calls can be chained.

Connect, then define namespaces and tables

```
catalog = Catalog(
    "metroride",                    # SQL prefix
    "lakehouse/metadata.ducklake",# catalog DB
    data_path="lakehouse/data")    # Parquet

catalog.create_namespace("silver")

schema = Schema.of(
    required("trip_id", StringType()),
    required("requested_at", TimestampTZType()),
    optional("fare_thb", DecimalType(10, 2)),
    required("service_date", DateType()),
)
silver = catalog.create_table(
    ("silver", "trips"), schema)
```

pyducklake type	Used for
`StringType()`	raw text, ids, zone
`IntegerType()`	source_row_number
`DecimalType(10, 2)`	fare_thb
`TimestampTZType()`	event times, ingested_at
`DateType()`	service_date

`required`

NOT NULL: inserting NULL fails

`optional`

NULL allowed (blank CSV cell)

Writing: append inside a transaction

```
rows = pa.table({"trip_id": [...], ...})

with catalog.begin_transaction() as txn:
    catalog.set_commit_message(
        "bronze: ingest trips_2026-04.csv",
        author="phase-1 tutorial")
    txn.load_table(table_id).append(rows)
```

Also available, not used in the tutorial:

```
table.delete("trip_id = 'X'")
table.overwrite(df)
table.upsert(df, join_cols=["trip_id"])
```



Input is a PyArrow table

Polars or any Arrow-compatible object also works.



Columns match by position

Build the Arrow table in schema order.



required rejects NULL

The append raises ConstraintException.



No commit, no snapshot

Failed or rolled-back writes leave no trace.

Reading: build a scan, then materialise it

```
table.scan("status = 'cancelled'")
```
→
```
.select("trip_id", "fare_thb")
```
→
```
.with_limit(5)
```
→ *one of*

Terminal method	Returns
`.count()`	number of rows
`.to_pandas()`	pandas DataFrame
`.to_arrow()`	PyArrow table, exact types
`.to_duckdb()`	DuckDB relation for more SQL

```
# or plain SQL, same connection
con = catalog.connection
con.sql("""
  SELECT count(*)
  FROM metroride.silver.trips
""").df()
```



pandas shows DECIMAL as float64 and DATE as datetime64. Check to_arrow().schema to see the stored types.

History: inspect, time travel, change capture

Inspect

```
t.snapshots()
t.current_snapshot()
t.inspect().snapshots()
t.inspect().files()
```

Lists snapshots with changes and commit messages, and the Parquet files of a table.

Time travel

```
t.scan()
 .with_snapshot(6)
t.scan()
 .with_timestamp(dt)
-- SQL
AT (VERSION => 6)
```

Read the table exactly as it was at an older snapshot or point in time.

Change capture

```
t.table_insertions(
    start_snapshot=7,
    end_snapshot=8)
t.table_changes(...)
```

Rows inserted (or deleted) between two snapshots. table_changes adds change_type.

Four SQL patterns the tutorial relies on



Anti-join

Keep only keys that are not loaded yet.

```
WHERE NOT EXISTS (
  SELECT 1 FROM silver.trips s
  WHERE s.trip_id = b.trip_id)
```



TRY_CAST

A bad value becomes NULL instead of an error.

```
TRY_CAST(requested_at
         AS TIMESTAMPTZ)
```



QUALIFY

Filter on a window function: one row per key.

```
QUALIFY row_number() OVER (
  PARTITION BY trip_id
  ORDER BY ingested_at) = 1
```



SHA-256 file hash

Same bytes give the same hash, so a reloaded file is detected.

```
hashlib.sha256(
  path.read_bytes()
).hexdigest()
```

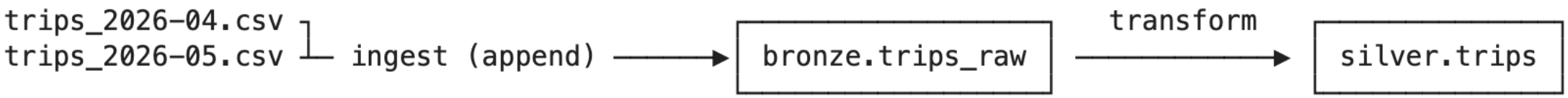
Part 3

Tutorial: Simple Medallion Architecture

tutorial.ipynb: April, then May, then back in time

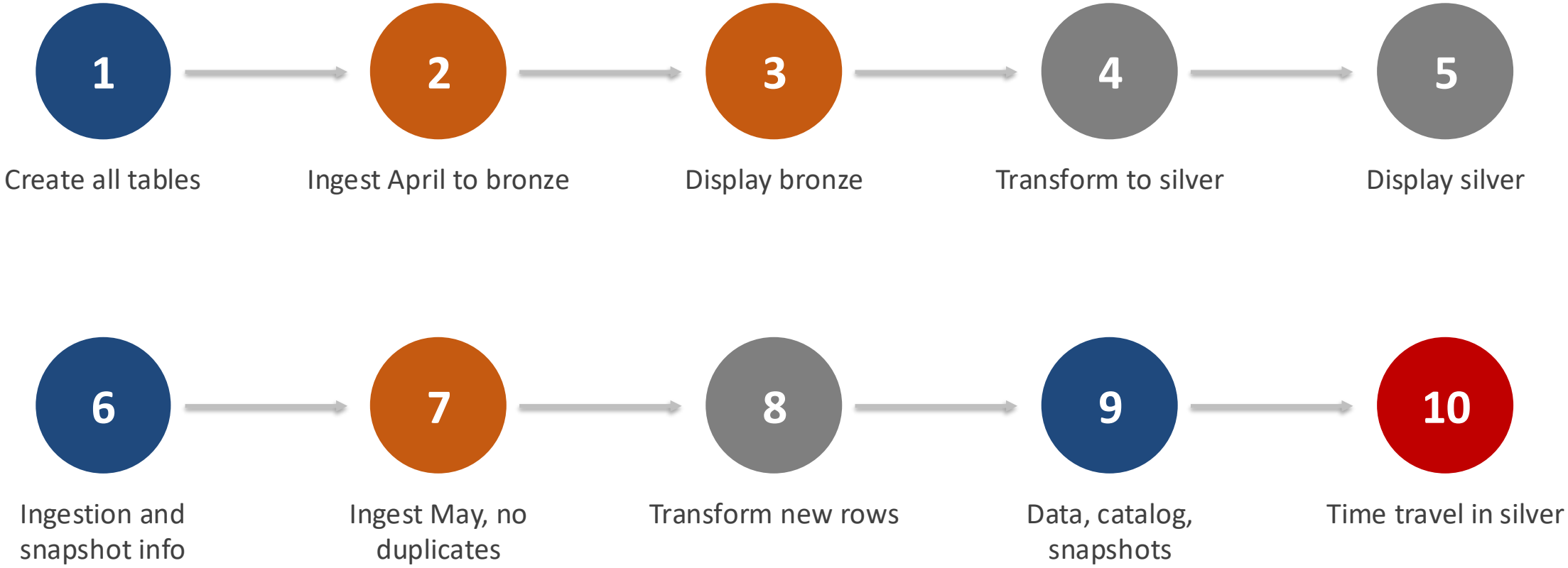


Data Pipeline



- Demonstrate how to data ingestion and data movement in data lakehouse using pyducklake
- CSV data sources – Bronze table – Silver table
- How to handle duplication and reject bad data quality

Ten steps, two layers, two months



CHULA ENGINEERING
Foundation toward Innovation

Step 1 · Create all tables

bronze.trips_raw

Columns	Type
10 source columns, trip_id … fare_thb	text, optional
source_file	text, required
source_row_number	integer, required
file_sha256	text, required
ingested_at	timestamptz, required

silver.trips

Columns	Type
trip_id, driver_id, service_zone, status	text, required
requested_at, accepted_at	timestamptz, required
picked_up_at, completed_at, cancelled_at	timestamptz, optional
fare_thb	decimal(10,2), optional
service_date	date, required
time_slot	text, required

```
catalog.create_namespace("bronze");  catalog.create_namespace("silver")
bronze = catalog.create_table(("bronze", "trips_raw"), bronze_schema)
```

Steps 2–3 · Ingest April with two duplicate guards

1

Hash the file

SHA-256 of the bytes

→

G1

File guard

Hash already in bronze? Skip the file

→

2

Read as text

All columns pa.string(); + 4 metadata columns

→

G2

Row guard

trip_id already in bronze? Drop the row

→

3

Append

One transaction with a commit message

132

April rows appended to bronze in snapshot 5

```
ingest_trips_csv(SOURCE_DIR / "trips_2026-04.csv")
# OK  trips_2026-04.csv: 132 rows read, 132 appended,
#     0 duplicates skipped
bronze.scan().with_limit(5).to_pandas()   # step 3
```

Steps 4–5 · Transform bronze to silver

1 Pick new rows

Bronze rows whose trip_id is not in silver (NOT EXISTS)

2 Parse and derive

TRY_CAST to timestamptz and decimal; service_date and time_slot in Bangkok time

3 Validate

Keep rows that pass every rule; one row per trip_id (QUALIFY)

Status	Validation rule
all	ids present · zone in Siam/Asok/Ari/Silom · requested ≤ accepted · request at 08:xx or 18:xx
completed	accepted ≤ picked_up ≤ completed · fare present and ≥ 0 · no cancelled_at
cancelled	accepted ≤ cancelled_at · no pickup, completion or fare

132

rows to silver, 0 rejected, snapshot 6. Trips: 124 completed, 8 cancelled.

Step 6 · Ingestion log and snapshots after April

Ingestion log (from bronze metadata columns)

source_file	file_sha256	rows	first_row	last_row
trips_2026-04.csv	ee6286bcdafff7d1…	132	1	132

Snapshots: bronze.inspect().snapshots()

id	changes	commit_message
0–2	schemas_created: main, bronze, silver	
3	tables_created: bronze.trips_raw	
4	tables_created: silver.trips	
5	tables_inserted_into: [3]	bronze: ingest trips_2026-04.csv
6	tables_inserted_into: [4]	silver: transform 132 new trips

Each table now has one Parquet file: data/bronze/trips_raw/… and data/silver/trips/… (inspect().files())

Step 7 · Append May without duplicate rows

+137

Nightly May load

New hash, new trip_ids: appended in snapshot 7

0

Replay of both files

File guard: both hashes already in bronze

0

Re-sent 3 May trips

New hash, but row guard finds every trip_id

bronze_rows	distinct_trip_ids	duplicate_rows
269	269	0

Re-runs that write nothing create no snapshot.

Steps 8–9 · Transform new data, inspect both layers

+137

May rows to silver in snapshot 8 (127 completed, 10 cancelled)

0

rows on a second run: the transform is idempotent

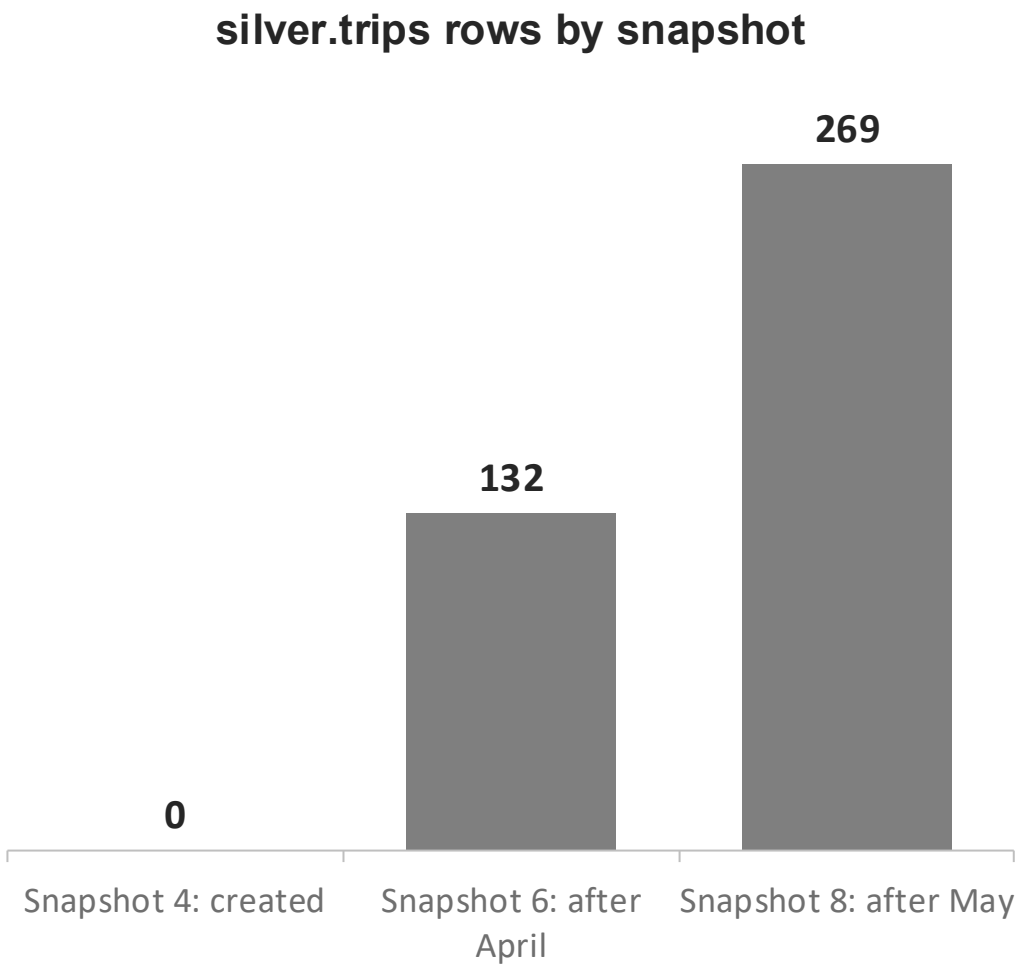
Rows per month

month	bronze.trips_raw	silver.trips
2026-04	132	132
2026-05	137	137

Data files from the catalog (ducklake_data_file)

table	rows	added in snapshot
bronze.trips_raw	132	5
silver.trips	132	6
bronze.trips_raw	137	7
silver.trips	137	8

Step 10 · Time travel in silver



By snapshot id

```
silver.scan().with_snapshot(6).count()    # 132
```

By timestamp

```
silver.scan().with_timestamp(
    time_after_april).count()             # 132
```

In SQL

```
SELECT count(*) FROM metroride.silver.trips
  AT (VERSION => 6)                          -- 132
```

What changed

```
silver.table_insertions(
    start_snapshot=7, end_snapshot=8)    # 137 rows
```

Wrap-up

Key ideas

- Catalog (SQL) plus data (Parquet); every commit is a snapshot
- Bronze keeps raw text and ingest metadata; two guards stop duplicates
- Silver is typed and validated; the transform is incremental and idempotent
- Time travel and change capture come for free from snapshots

Try it yourself

1. Ingest a copy of April with one bad row; check it reaches bronze but not silver
2. Time travel bronze.trips_raw to before the May file was ingested
3. Build gold.daily_service_performance from silver.trips